

## Project 2 Report

### Administrative

Team Name: fart

Team Members + Github user Names:

- Christina Maribay - christinamaribbay
- Rohan David - rohancdavid
- Marina Ma - moonmara

Link to Github repo: <https://github.com/christinamaribbay/rivals-aggression-analyzer.git>

Link to video demo: <https://youtu.be/GD13tnGKHxo>

### Extended and Refined Proposal (2 pages)

- **Problem:**
  - Determine whether there is a correlation between in-game chat aggression and performance metrics (defined by in-game rank) in Marvel Rivals.
- **Motivation:**
  - Aggression is a common result of “tilting”, where a person experiences emotional dysregulation as a result of unfavorable outcomes in games. Tilting often negatively impacts a player’s performance. Having a quantifiable score helps determine whether hostile behavior is a trait of skilled players or a detriment to winning.
- **Features Implemented:**
  - Get average aggression scores per rank
  - Get aggression score for a custom message
  - Compare splay and hash performance with existing data
- **Description of Data:**
  - Generated data:
    - String comment made by player in match
    - String rank corresponding to in-game rank of player that made the comment (bronze, silver, gold, platinum, diamond, grandmaster, celestial, eternity, one above all)
- **Tools/Languages/APIs/Libraries Used:**
  - Tools: Clion, GitHub
  - Language: C++
  - APIs: n/a
  - Libraries:
    - <string>
    - <vector>
    - <map>

- <iostream> (for printing)
  - <fstream> & <sstream> (for data parsing)
  - <chrono> (for performance comparison)
  - <iomanip> (for rounding up float results for printing)
  - <algorithm> & <random> (for generating data)
- **Algorithms Implemented:**
  - Splay Tree
    - left rotation, right rotation, splay operations, insert, search, clearing
  - Hash Table
    - Insert, search, string hashing, check if empty
- **Additional Data Structures/Algorithms used:**
  - <map> used to calculate aggressionScores by rank
- **Distribution of Responsibility and Roles:**
  - Christina Maribbay:
    - Main
    - Data generation
  - Rohan David:
    - Player class
    - Splay tree
  - Marina Ma:
    - Hash table
    - Presentation video

### Analysis (1.5 pages)

- Any changes the group made after the proposal? The rationale behind the changes.
  - We decided to change the trie to a hash because it was a better direct comparison to the splay tree. Since the search functions needed to return the aggressionScores of each stored word, it was easier to compare a hash with nodes that contain both string word and float aggressionScores rather than using a trie which would only store each word.
- Big O worst case time complexity analysis of the major functions/features you implemented.
  - Player class
    - Overall worst case complexity is  $O(m)$  where  $m$  is the length of the chat message because of `getChatMessage()` having the highest time complexity.
    - `getRank()` -  $O(1)$
    - `getChatMessage()` -  $O(m)$ , where  $m$  is length of chat message
    - `getChatAggressionScore()` -  $O(1)$

- `setRank()` -  $O(r)$ , where  $r$  is length of rank string
  - `setChatMessage()` -  $O(m)$
  - `setChatAggressionScore()` -  $O(1)$
- Splay Tree
  - Overall worst case time complexity for a major tree operation is  $O(n)$  where  $n$  is the number of nodes in the tree.
  - `rotateLeft()` and `rotateRight()` -  $O(1)$
  - `splay()`, `insertHelper()`, `insert()`, `search()`, `clearHelper()`, `clear()`, `destructor` are  $O(n)$
- Hash Table
  - Overall worst case time complexity for any given operation (e.g. insertion, deletion, search) is  $O(n)$  where  $n$  is the number of slots in the table.
  - `isEmpty()`, `insertWord()`, and `searchWord()` are all  $O(n)$ .
  - `hashFunction()` is  $O(m)$  where  $m$  is the length of the key.

## Reflection (1- 1.5 pages)

- As a group, how was the overall experience for the project?
  - Overall it was a great experience collaborating on a shared interest while applying it to data structures and what we've been learning. Splitting the workload and cooperating went smoothly. We were able to efficiently work through our own parts and mend them together to finalize the project.
- Did you have any challenges? If so, describe.
  - Figuring out how to measure and rank aggression by chat message.
  - Choosing the different data structures that would work for our case and comparing them.
  - The Main loop had a couple initial bugs involving infinite loops and incorrect if-else branching, but it was simple to debug.
  - There were some compilation errors regarding the configurations to run the program while pulling the final project but it was sorted in the end.
- If you were to start once again as a group, any changes you would make to the project and/or workflow?
  - Generally no - the work was delegated evenly and our individual components came together seamlessly. Perhaps there could be many more key words (ex. Toxic words, safe words, neutral words) so that the aggressionScore calculation is more precise.
- Comment on what each of the members learned through this process.
  - Christina Maribbay:
    - Learned how to use <algorithm> and <random> to randomly generate data. I did not realize that the chatMessage data did not have to be grammatically correct sentences for the aggressionScore calculation to work.
  - Rohan David:
    - Gained a better understanding of how splay trees work such as rotations and operations and how to utilize that for our own project.
    - Creating tree data structures from scratch and having to implement our own data into our program.
  - Marina Ma:
    - Implementing the hash table from scratch gave me a better understanding of how hash tables worked conceptually. It was a little confusing to understand regarding how to handle collisions and the hashing functions without being hands on.

## References

- [1] J. H. Goldstein, *Sports, Games, and Play*. Psychology Press, 2012.
- [2] Moreau, A., Bethencourt, A., Payet, V., Turina, M., Moulinard, J., Chabrol, H., & Chauchard, E. (2024). Rage in video gaming, characteristics of loss of control among gamers: A qualitative study. *Psychology of Popular Media*, 13(3), 331–337.  
<https://doi.org/10.1037/ppm0000481>